

Bài thực hành 03

Lập trình C An toàn: Quản lý Bộ nhớ & Đánh giá Rủi ro

Hệ thống SecureSys – Module Inventory

Học phần CSE703093 – An toàn phần mềm
Phenikaa University

Ngày 8 tháng 7 năm 2026

Tóm tắt nội dung

Bài thực hành tái hiện ba lớp lỗi bộ nhớ kinh điển trong C – **Buffer Overflow** (CWE-121), **Use-After-Free** (CWE-416), **Memory Leak** (CWE-401) – phát hiện bằng **AddressSanitizer**, sau đó áp dụng một **mô hình đánh giá rủi ro** đơn giản hóa theo tinh thần CVSS để định lượng mức độ nghiêm trọng và ưu tiên khắc phục.

Mục lục

1 Kiến thức nền tảng	1
2 Ba lỗi hỏng minh họa	1
3 Kết quả phát hiện bằng ASan	2
4 Đánh giá rủi ro (Risk Assessment)	2
5 Kết quả kiểm thử tự động	2
6 Tài liệu tham khảo	3

1 Kiến thức nền tảng

AddressSanitizer (ASan) là công cụ phân tích động (dynamic analysis) chèn thêm mã kiểm tra vùng nhớ (redzone) quanh mỗi khối cấp phát, giúp phát hiện truy cập ngoài biên và sử dụng sau giải phóng ngay tại thời điểm chạy. Điểm mấu chốt sự phạm: ASan chỉ chèn redzone ở **biên vùng cấp phát** – tràn bên trong một struct (ghi đè lên field kế bên) có thể **không** bị phát hiện.

2 Ba lỗi hồng minh họa

CWE	Tên	Vị trí / Nguyên nhân
CWE-121	Buffer Overflow	strcpy() không kiểm tra độ dài, ghi vượt vùng heap cấp phát 32 byte
CWE-416	Use-After-Free	Truy cập con trỏ sau khi đã free()
CWE-401	Memory Leak	Vòng lặp cấp phát lặp lại nhưng thiếu free()

```

1 int add_book_title_SAFE(Book *b, const char *title) {
2   if (b == NULL || title == NULL) { ... return -1; }
3   int written = snprintf(b->title, TITLE_MAX, "%s", title);
4   return (written >= TITLE_MAX) ? 1 : 0; /* bao truncation */
5 }

```

Listing 1: Bản vá lỗi CWE-121 (trích secure_inventory.c)

3 Kết quả phát hiện bằng ASan

Kịch bản	Bản vulnerable	Bản secure
overflow (payload dài)	heap-buffer-overflow	Sạch (chuỗi bị cắt bớt an toàn)
uaf	heap-use-after-free	Sạch
leak	memory leaked (LeakSanitizer)	Sạch

4 Đánh giá rủi ro (Risk Assessment)

Áp dụng mô hình CVSS-lite (đơn giản hóa, **không** phải công thức CVSS chính thức):

$$\text{BaseScore} = \min(\text{Impact}_{\text{sub}} + \text{Exploitability}_{\text{sub}}, 10)$$

ID	CWE	Điểm	Mức độ	
VULN-001	CWE-121	9.8	CRITICAL	Buffer Overflow
VULN-002	CWE-416	6.9	MEDIUM	Use After Free
VULN-003	CWE-401	5.4	MEDIUM	Memory Leak
VULN-004	CWE-476	2.5	LOW	NULL Pointer Dereference (tiềm ẩn)

Thứ tự ưu tiên khắc phục: VULN-001 → VULN-002 → VULN-003 → VULN-004, khớp trực giác: lỗi cho phép ghi đè bộ nhớ tùy ý (buffer overflow) luôn nghiêm trọng nhất vì có thể dẫn tới thực thi mã tùy ý.

5 Kết quả kiểm thử tự động

tests/test_memory_safety.py thực hiện 8 test (3 phát hiện lỗi ở bản vulnerable, 3 xác nhận sạch ở bản secure, 2 kiểm tra đánh giá rủi ro). **Kết quả: 8/8 test PASS.**

Mục tiêu học tập

- Tái hiện và phát hiện 3 lớp lỗi bộ nhớ bằng AddressSanitizer.
- Sửa lỗi theo nguyên tắc lập trình C an toàn.
- Áp dụng mô hình đánh giá rủi ro định lượng (CVSS-lite).
- Hiểu giới hạn của ASan (tràn trong struct không luôn bị bắt).

Yêu cầu hoàn thiện

1. Bảng liệt kê 3 lỗi kèm stack trace rút gọn.
2. Giải thích hiện tượng tràn trong struct không bị ASan bắt.
3. Báo cáo đánh giá rủi ro đầy đủ, kèm 1 lỗ hổng tự thêm (VULN-005).
4. Log make test toàn bộ PASS.

Yêu cầu kết quả

ASan phát hiện đủ 3/3 lỗi trên bản vulnerable, 0/3 lỗi trên bản secure; tests/test_memory_safety.py toàn bộ 8 test PASS.

6 Tài liệu tham khảo

- Đề cương CSE703093 – An toàn phần mềm, Chương 2 (2.1, 2.3).
- CWE-121, CWE-416, CWE-401, CWE-476 (cwe.mitre.org).
- FIRST.org – CVSS v3.1 Specification.